

```

(gdb) disas main
Dump of assembler code for function main:
0x0000000000405308 <@>:      push    %rbp
0x0000000000405311 <+1>:      mov     %rsp,%rbp
0x0000000000405334 <+4>:      mov     $0x4005e0,%edi
0x0000000000405339 <+5>:      callq  0x400410 <puts@plt>
0x000000000040533e <+6>:      nop
0x000000000040533f <+7>:      nop     $0x0,%eax
0x0000000000405344 <+8>:      pop     %rbp
0x0000000000405345 <+9>:      retq

End of assembler dump.

(gdb)

```

nop replaced with int3

Figure 7: Comparison of the runtime images of the ELF before and after placement of probe point (output obtained using *gdb disas*)

The screenshot shows a terminal session where kernel debugging is configured. The user sets environment variables for kernel debug type and current tracer, then enables tracing events for user enable and tracing on. A cat command displays the kernel's response, indicating that tracing is enabled. Finally, the user runs the 'tracert' command, which produces a trace output showing task details like PID, CPU, delay, and function.

```
hemant@hemant:~/home/hemant$ sudo sysctl -w kernel.debug.type=debugfs
kernel.debug.type = debugfs
hemant@hemant:~/home/hemant$ sudo sysctl -w kernel.debug.current_tracer=tracert
kernel.debug.current_tracer = tracert
hemant@hemant:~/home/hemant$ sudo sysctl -w kernel.debug.tracing/events/user/enabl
kernel.debug.tracing.events.user.enable = 1
hemant@hemant:~/home/hemant$ sudo sysctl -w kernel.debug.tracing/tracing/on
kernel.debug.tracing.on = 1
hemant@hemant:~/home/hemant$ cat /sys/kernel/debug/tracing/trace_events
#user/in main /test-0x8000000000000035
hemant@hemant:~/home/hemant$ cat /sys/kernel/debug/tracing/trace
#tracer : nop
hemant@hemant:~/home/hemant$ entries-in-buffer/entries-written: 0/0 #P:4
```

TASK-PID	CPU#	delay	FUNCTION
			irqs-off
			need-resched
			hardirq/software_irq
			preempt-death
			delay

```
hemant@hemant:~/home/hemant$
```

Figure 8: Steps to use *uprobes* interface *trace* file is empty

events mentioned above take place.

Tracing without Systemtap

Now, let's trace and probe manually using *uprobes* directly!

First, find out the mount point of *debugfs*. This virtual subsystem exists only in memory. We can find that out by using:

```
$ mount | grep debugfs
```

In Figure 8, we find that it is `/sys/kernel/debugfs`.

This contains a *tracing* directory and there are a few available tracers. We need the *nop* tracer.

```
# echo nop > /sys/kernel/debug/tracing/current_tracer
```

In one terminal, let's find out the location of the marker we want to trace using the same *readelf*.

Let's suppose it is 0x40053e. Feed this address to `uprobe_events` in the tracing directory:

```
# echo "p:user/in_main ./test:0x53e" > /sys/kernel/debug/
```

[tracing/uprobe_events](#)

Note that we are probing on 0x53e and not on 0x40053e. The `.text` section in x86 starts at 0x400000 and this is taken care of by the `uprobes` interface. So just subtract 0x400000 from the note location we retrieved using `readelf`. In the context of `uprobes`, user is an event group and `in_main` is an event.

After making this entry, a directory named *user* is created in the *events* subdirectory inside the *tracing* directory. This corresponds to a group of events, of which, one event is *in_main*.

Clear out the *trace* file:

```
# echo > /sys/kernel/debug/tracing/trace
```

Let's enable the event we want to trace, as follows:

```
# echo 1 > /sys/kernel/debug/tracing/events/user/enable
```

In this case, we are enabling the complete event group which doesn't make a difference.

Start tracing, as follows:

```
# echo 1 > /sys/kernel/debug/tracing/tracing_on
```

The tracing has begun. Now, just run the program in another terminal and then let's look at the trace:

```
# cat /sys/kernel/debug/tracing/trace
```

It shows the task, its PID, the timestamp when the hit was recorded and the event name we are trying to trace. It also shows the hits (refer to Figure 9).

The arguments' support is not yet present in the kernel. But this support is being integrated and will soon be available, presumably in the next kernel version.

This article will just familiarise you with SDT markers and their importance, giving you a little insight into their probing and tracing. It's up to you as to how you use them. Their importance can be gauged by the fact that QEMU uses over 500 markers. They can also be used in network programs to see the data received at the client's or at the server side. Libvirt uses them before making the RPC calls, so that if we probe on these markers, we will be able to know the data being sent and received. If you face any problem, just probe them, find out the relevant data and find out the source.  **END**

By: Hemant Kumar Shaw

The author is a Linux user, an open source enthusiast and a contributor.